

Research Topics in Recommender Systems

INFO345

24.11.2025

Pages: 20

Candidate numbers: 131, 134

Table of contents

1. Abstract **3**

2. Introduction **3**

2.1 Background *3*

2.2 Motivation *3*

2.3 Research question *4*

3. Data Analysis **4**

3.1 Items (Beer Styles and Products) *4*

3.2 User Reviews and Behavior *5*

3.3 Item Popularity Analysis *8*

4. Developing a Recommendation Model **9**

4.1 Model selection and data preparation *9*

4.2 Libraries used *10*

4.3 Threshold Justification *10*

4.4 Models Overview *11*

4.5 Train/Test Split *11*

4.6 Evaluation Metrics *12*

4.7 Grid Search *13*

4.8 Hybrid Model *13*

5. Evaluating the Recommendation Models **13**

5.1 Offline Evaluation *13*

5.2 Error Distribution Analysis *13*

5.3 Predicted Rating Behavior of the Models *14*

5.4 Overall Performance Comparison *15*

5.5 Evaluating the Hybrid Model *17*

The model 17

Hybrid performance 17

Interpretation 18

6. Summary of Findings **19**

7. Limitations **19**

8. Future Work **20**

9. References **20**

1. Abstract

We are surrounded by data, and the volume is constantly growing faster than anyone can physically process. Recommender systems therefore play a central role in helping users navigate large item catalogues. In this project we explore the challenge of building a beer recommendation system using the large-scale and highly sparse BeerAdvocate dataset, which contains more than 1.5 million user reviews. This dataset presents unique difficulties due to strong popularity imbalance, long-tail item distributions, rating behaviour that is heavily skewed towards the head of the distribution, and more. We develop and evaluate several collaborative filtering techniques; SVD, Co-Clustering, KNNBaseline, alongside two baseline models. We conduct a substantial amount of offline evaluation using accuracy metrics, top-N metrics, coverage measures, diversity measures and runtime performance. The study will leave us with insights to the strengths and limitations collaborative filtering models have on highly imbalanced rating data, and also demonstrates how such models can broaden user preferences beyond familiar beer choices.

2. Introduction

2.1 Background

Modern platforms like Netflix, Amazon and Spotify rely on recommender systems to help users navigate the overwhelming volumes of content. It is useful for a wide range of purposes, such as selecting movies, music, products, and so on. Users are increasingly dependent on algorithms to filter information and present items that match their tastes. Beer recommendation is no different. Unlike movies or books, what beer enjoyers choose often revolves around sticking to brands they already know. Most people drink only a narrow set of beers, even though we have thousands of varieties. According to the National Beer Wholesalers Association (2023), the top ten brewers account for nearly 88% of the U.S market sales, proving that the industry is dominated by popularity and habit rather than exploration and curiosity.

2.2 Motivation

The dataset from BeerAdvocate is large, sparse, and extremely imbalanced. User activity is long-tailed, meaning most users only rate a handful of beers. Item popularity is long-tailed, meaning a small number of beers receive the majority of reviews. Ratings are skewed upward, where over 60% of all ratings are 4.0 or higher (rating scale 0-5).

These properties are each a challenge in themselves, making it difficult for models to learn meaningful representations, give confident recommendations, without simply defaulting to the most popular beers. We are especially interested in encouraging users to broaden their tastes for beer,

make them discover new beers they haven't tried before, based on personalised recommendations. This requires that our models have a certain degree of accuracy, while also providing diversity, novelty and coverage across the item catalog.

2.3 Research question

Based on these challenges, we have formulated the following research question:

How well can collaborative filtering methods recommend new and diverse beers to users, beyond their typical choices, when trained on a highly sparse and imbalanced dataset?

To answer this, we will develop several collaborative filtering models and evaluate them against random and popularity-based baselines, using a wide range of offline metrics.

3. Data Analysis

Here is a link to the dataset that we are using for the project:
https://www.kaggle.com/datasets/rdoume/beerreviews

3.1 Items (Beer Styles and Products)

Most of the reviewers from the dataset appear to be beer enthusiasts, giving us a lot of uncommon beers as the most reviewed. Consequently, the beers that are widely available in grocery-stores are not necessarily the most frequently reviewed ones. Figure 1 illustrates the five most reviewed beers across the whole dataset.

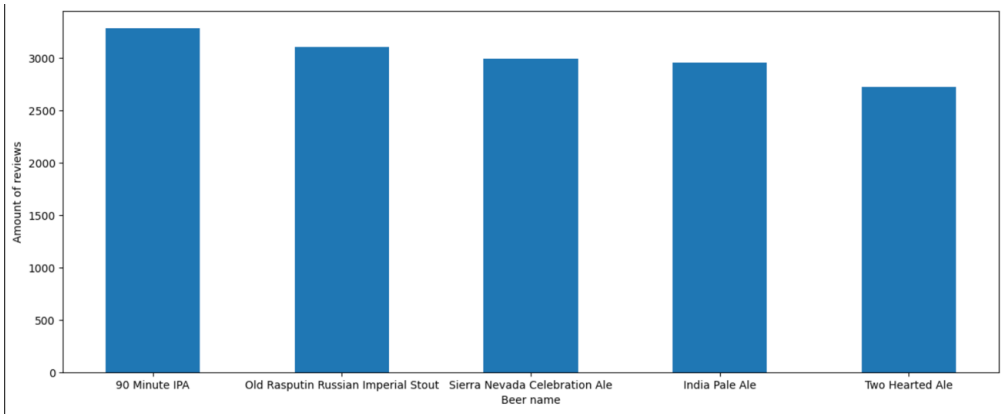


Figure 1: Top five most reviewed beers

Figure 2 shows how many styles each user has reviewed. Overall, there exist 104 different beer styles and 32,908 different users. Most users have reviewed very few styles of beers, creating a long-tailed graph. This supports our claim that beer-drinkers often will stick to the styles they already like. On the other hand, there also exists a consistent number of users that have reviewed a large variety of styles. There are even 17 users that have reviewed all 104 styles.

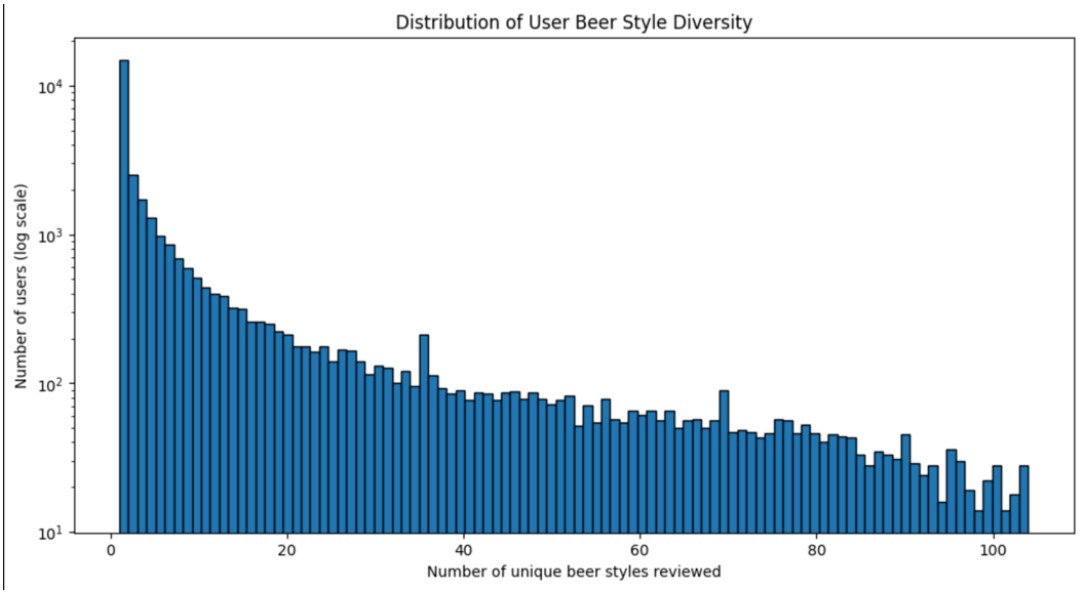


Figure 2: Distribution of user beer style diversity

Finally, we inspect rating frequency per beer style. Figure 3 displays the 20 most reviewed beer styles, showing the strong popularity imbalance in the dataset. A small portion of beer styles accumulate the majority of all ratings.

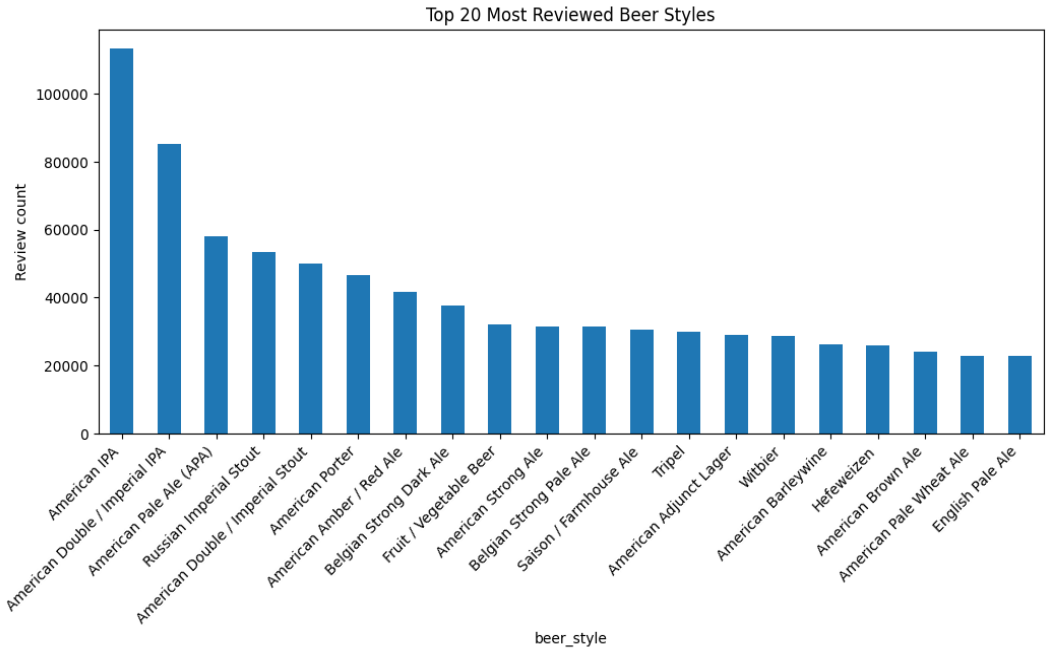


Figure 3: Top 20 Most Reviewed Beer Styles

3.2 User Reviews and Behavior

Next, we analyze the users. Understanding user behaviour is crucial, especially with collaborative filtering models relying on patterns across different users.

Figure 4 shows the distribution of overall ratings from the reviewers. The median rating is 4 and the mean is 3.81, suggesting that most users give high ratings. This is important to consider when choosing a threshold for what we deem as a “positive” rating.

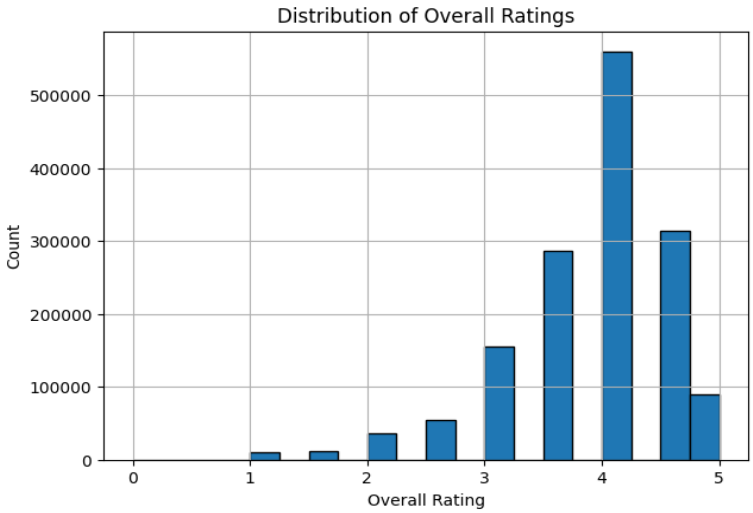


Figure 4: The distribution of overall ratings

To understand how active users are, Figure 5 illustrates the distribution of the number of reviews per user. The dataset contains 32,908 users, but we can see that most of them have reviewed very few beers. The median number of reviews per user is 3, while a small number of extremely active users reviewed hundreds or even thousands of beers.

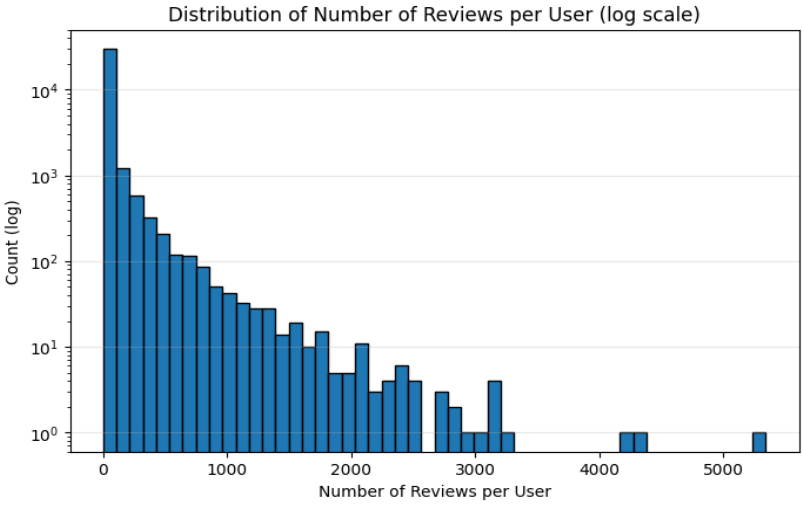


Figure 5: Distribution of number of reviews per user

Key statistics:

- Total users: 32,908
- Median reviews per user: 3

- Average reviews per user: 46.14
- Maximum reviews by a single user: 5,346
- 95th percentile: 225 reviews
- 99th percentile: 816 reviews

To choose a rating threshold for defining “positive” reviews, we will examine the empirical cumulative distribution function (ECDF) in Figure 6

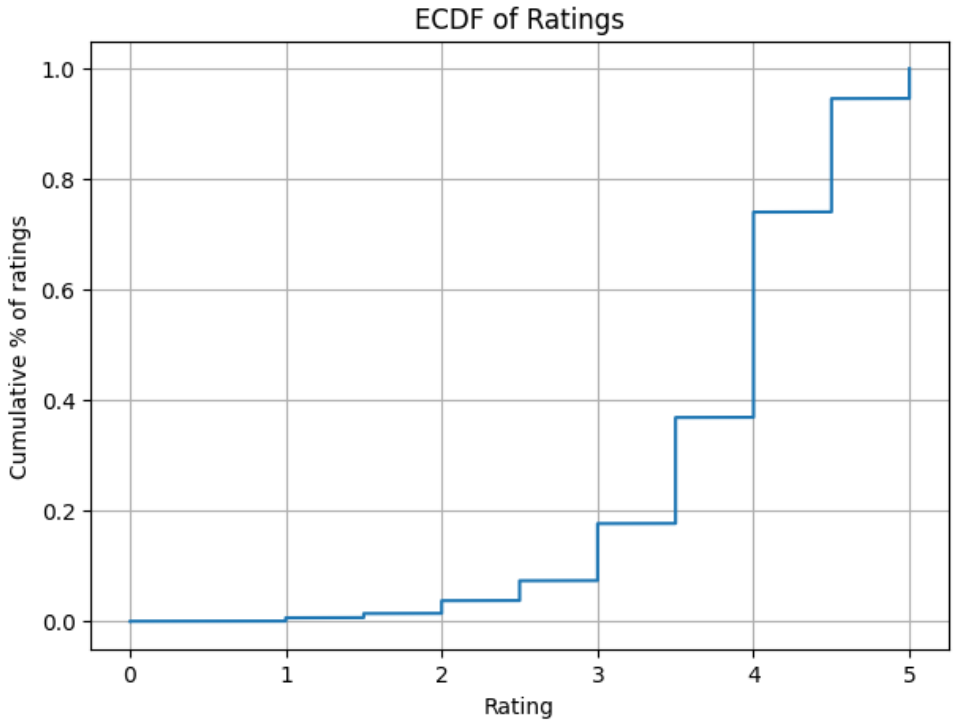


Figure 6: ECDF of ratings

Key statistics:

- Median rating = 4.0
- 63.1% of ratings are ≥ 4.0
- 82.3% of ratings are ≥ 3.5
- 26% of ratings are ≥ 4.5

A pattern worth highlighting is the strong positive skew in the rating distribution. Based on this distribution, we set 4.0 as the positive rating threshold for evaluation.

3.3 Item Popularity Analysis

We examine item popularity, which is critical for understanding the long-tail effect, which is very much present in our dataset. Figure 7 shows the distribution of number of reviews per item. The pattern we saw in users is also present here, most items receive very few reviews.

As the original dataset contains thousands of individual beers, many of them has only a few amount of reviews. To reduce sparsity and make the representation of items consistent, without collisions such as using the same name for different beers or using the same style, we created brewery_style. This was created by merging the ID of each brewery with the beer style. This creates a balance where some beers have now been grouped, while we still preserve distinctions between different breweries and styles. brewery_style has around 25% less items than the beer column with 33072 items in total. We will now do some item-level analysis on brewery_style.

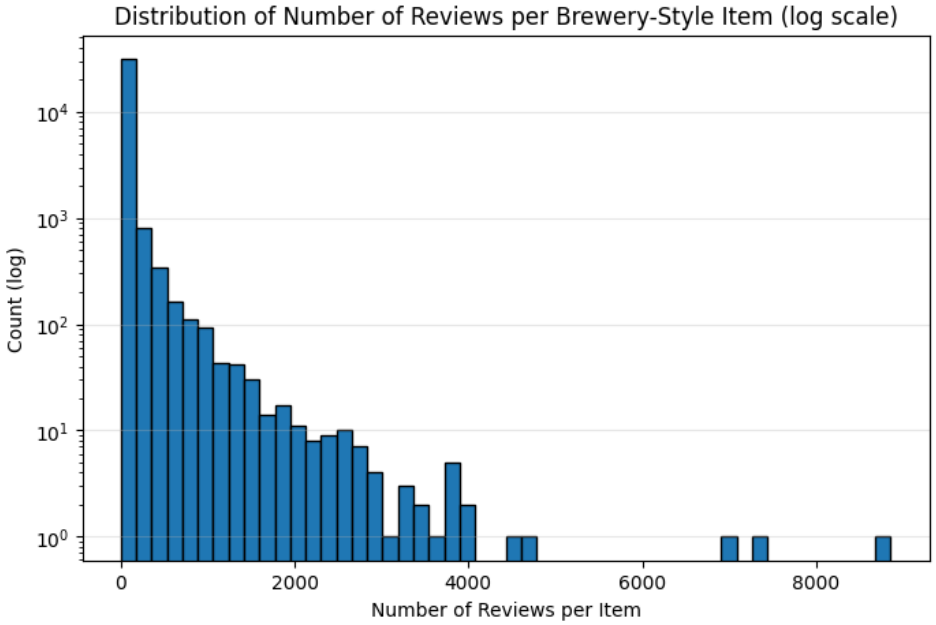


Figure 7: NoR per Brewery-Style item

Key statistics:

- Total items: 104 beer styles, but 33,072 Brewery-Style combinations
- Median reviews per item: 4.0
- Average reviews per item: 43.99

Figure 8 presents the cumulative distribution of item popularity. The top 5% of items account for a large proportion of all ratings, which does demonstrate that we have a very strong head-heavy distribution.

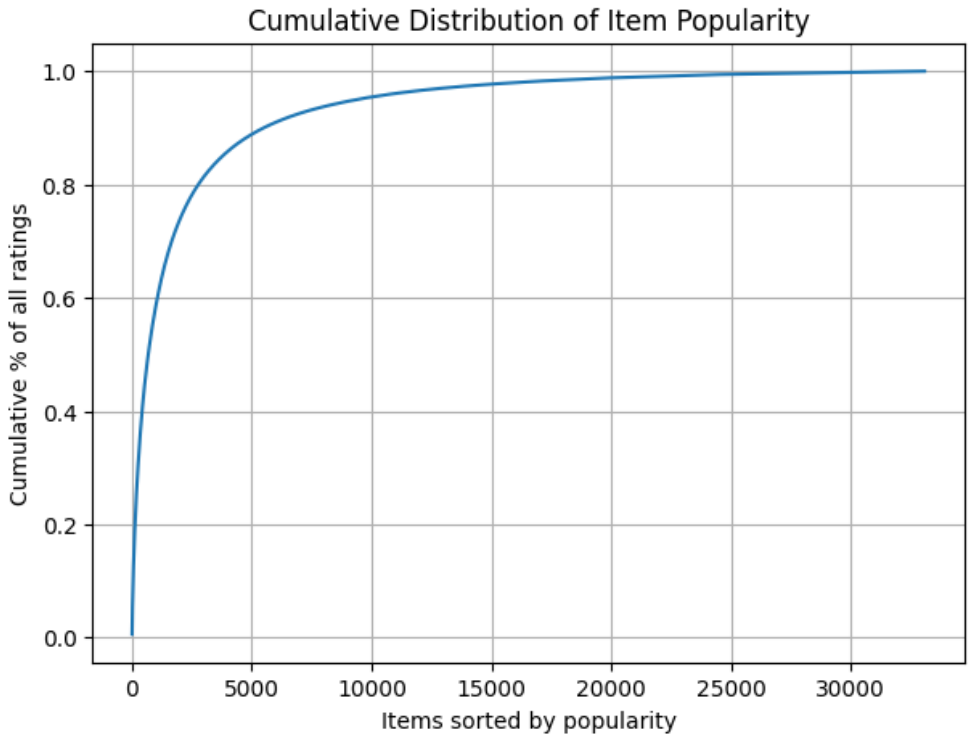


Figure 8: Cumulative Distribution of Item Popularity

This imbalance is very important in our case because popularity-based models will naturally perform well. Collaborative filtering might struggle a bit with recommending unpopular items, especially in cold start cases. We will therefore add coverage metrics for evaluation, as it is important to detect any popularity biases that might occur.

4. Developing a Recommendation Model

4.1 Model selection and data preparation

Our dataset contains a large number of explicit rating data and not a lot of descriptive features or metadata. According to Recommender Systems: An Introduction (Dietmar J, 2011, p.3), "Pure CF approaches do not exploit or require any knowledge about the items themselves." Based on this, as well as the structure of the dataset, we decided to build a Collaborative-Filtering based Recommender System. Collaborative filtering gives recommendations based on the preferences of similar users.

4.2 Libraries used

For the development of the project, we have used the following libraries:

- **Pandas:** Pandas is used to load and preprocess our dataset from a csv file. The DataFrame structure makes it easy for us to clean, explore and finalize the data before feeding it into a recommender system. Version: 2.3.3
- **Numpy:** Numpy is a library used to handle mathematical computations. It is also used to format data into arrays that are suitable for our models. Version: 1.26.4
- **Mathplotlib:** Mathplotlib is used to perform exploratory data analysis. It is especially useful when dealing with large datasets, because it helps us discover biases and outliers in the data. Version: 3.10.7
- **Scikit-learn:** We used scikit-learn's model selection tools in order to split the data into a train-set and test-set. We will use these sets to train and evaluate the performance of the models. Version: 1.7.2
- **Surprise:** Surprise offers a lot of ready-to-use CF algorithms that we will test out to try and find some that fit our data. Version: 1.1.4
- **Collections and time:** Collections module is used to store model predictions for each user and time module is used to measure training and evaluation runtime.

4.3 Threshold Justification

For our models we need to define a threshold that will evaluate items as either positive or negative. From section 3.2 we learned that overall rating is heavily skewed towards the upper end of the scale. Percentile analysis shows that 63.1% of all ratings are 4.0 and above. The ECDF of Ratings (Figure 4) also highlights a very significant jump specifically at the 4.0 mark. We interpret that users frequently choose this value to signal a generally positive experience. Also, to determine an appropriate threshold value we compared how many ratings would be considered positive at different cutoffs: 3.5 would make 82.3% of ratings positive, 4.0 would make 63.1% of ratings positive and 4.5 would make 26.0% of ratings positive. Using 3.5 as the threshold, thus making over 80% of ratings seems too permissive. The vast majority of beers would be considered to be positive. On the other hand, 4.5 seems too strict. Only the top 26% of ratings would be considered positive, which leaves the models less to work with in terms of signals between positive and negative. We consider 4.0 as a nice middle ground. It is the median rating, and 63.1% of ratings would be positive which is not as heavily skewed as the other threshold options. It also aligns with our view on ratings where 4.0 typically indicates a "good" beer.

The next threshold we need to decide is the minimum number of reviews a user needs to have in order to be able to be given a recommendation. The distribution of the number of reviews per user is heavily skewed. The median sits at 3, while the average is 46. This means that there are a relatively small number of extremely active users that pull the average up. Users with few reviews often have ratings that do not reflect their long-term tastes. Our CF models will therefore struggle to give accurate recommendations to these users. Because of this, we decided to set the threshold to ≥ 11 reviews.

4.4 Models Overview

We explored multiple CF models in the surprise library, among these were KNNBaseline, Co-clustering and SVD.

We tested both user-user and item-item based KNN, but in both cases they turned out to be computationally heavy because KNN requires to compute up to n^2 similarity scores. Additionally, KNN is also not able to learn hidden factors, it only looks at direct overlaps between items, while models like SVD is able to capture latent preference factors. KNN also tends to have a “popularity bias”; the items with the most ratings will get recommended the most, while less rated items will often be ignored even when they might be relevant. This bias does not align with our motivations for the project.

Co-clustering groups users and items into multiple clusters and makes predictions based on the cluster's averages. Because everyone inside the cluster gets recommended the same items, the model will typically give very “safe” and “general” recommendations to its users. This model also suffers from the popularity bias we find in KNN.

SVD is able to reduce the dimensionality of our data. It compresses the user-item matrix into three smaller matrices that help discover the relationship between the items and users (Tam, 2021). SVD makes predictions based on latent factors, hidden relationships between items and is not so reliant on direct overlaps. This makes the algorithm less prone to having a popularity bias and gives better overall coverage, which is desirable for our goals for the project.

4.5 Train/Test Split

The systems must of course be evaluated on unseen data for us to get a realistic estimate of performance in the real-world. Therefore, we split our dataset into a training set (75%) and a test set (25%). Because Surprise internally converts user and item IDs to integer IDs (Hug, 2015), the test set must satisfy that every user and every item in the test set must also appear in the training set. To ensure this we decided to implement a custom train/test using Reader and Dataset APIs from

Surprise. Now the models are never predicting ratings for unseen users or items. We also decided to apply that users must have ≥ 11 reviews before splitting the data, which ensures that every user in training has enough history to actually support collaborative filtering to be performed on the data.

4.6 Evaluation Metrics

After training a model we will evaluate the performance using a wide range of metrics that will capture accuracy, ranking quality, and diversity/coverage. All evaluation is strictly performed offline. First, we compute regression accuracy by using RMSE and MAE. These measure how close predicted ratings are to the true ratings in value. It is important to note that models like ours that are optimised on RMSE and MAE might often struggle with precision in recommendation, because predicted ratings naturally will lean towards the mean in order to preserve RMSE/MAE scores (Dietmar J., 2011, p.180)

Using the 4.0 threshold for positive/negative ratings we can compute classification-style metrics. Precision is the proportion of recommended items that were truly liked by a user. Recall shows the proportion of liked items that were successfully recommended for a user. F1 is a harmonic mean of precision and recall. And since in real recommender systems, we will only recommend a handful of items we also implement Precision@5 where we see how many of the top 5 predicted items the user liked.

For coverage metrics we implement two notions of coverage. First, we have User Coverage which is the fraction of users where the model could provide at least one positive-scored recommendation (Cañamares, R., 2020). Of course we could also let the models recommend the highest predicted items for a user even if it is under the threshold of 4.0, but we decided to implement it like this so we can see if the model can give confident recommendations to a high amount of users, and not just a minority.
We also have Conditional Coverage, which measures how many different items the model ever recommends that it ever predicted above the threshold. This helps us evaluate how much the model is willing to explore different options, or if it sticks to recommending the same beers to everyone.

To evaluate diversity of recommendations we compute Intra-List Similarity (ILS) for each user's top-K list. A lower ILS means a more diverse list (Jesse M., 2023). This is very important for our research question (specifically "broaden a user's beer taste"), because we want to avoid recommending near-identical beers.

To evaluate practicality and scalability we also record Training time and Prediction time. Using these records, we can see how heavy a model is to run, and decide on which model to use if results are near identical.

4.7 Grid Search

In order to select good hyperparameters for the different models, we performed grid-search optimisation by using GridSearchCV from the Surprise library. RMSE was used as the optimisation criteria because Surprise uses grid-search around rating-prediction accuracy. (Hug. N., 2015)

4.8 Hybrid Model

Even if we saw that SVD consistently achieved a strong overall performance, we wanted to see if we could further improve it by combining it with another model. SVD's biggest struggle was to provide recommendation for users with very few reviews, resulting in a slightly reduced user coverage. The popularity-based recommender can always make predictions, even for users with few reviews. To take advantage of the strengths of both approaches we wanted to experiment with a very simple hybrid model. Users with 11-45 reviews receive popularity-based predictions, while users with more than 45 reviews receive SVD predictions, since SVD can make very accurate predictions from rich user histories. We will also evaluate this hybrid model alongside the other models in section 5.

5. Evaluating the Recommendation Models

5.1 Offline Evaluation

To evaluate our models, we have decided to perform an offline evaluation with the evaluation metrics we discussed in section 4.6. We have chosen to do an offline evaluation rather than an online evaluation because it allows us to test and analyze our recommender system without needing feedback from real users. Figure 9 visualizes the absolute error distributions for each model. Figure 10 and Figure 11 are tabular comparisons of all relevant metrics.

5.2 Error Distribution Analysis

In order to understand how each model behaves in greater detail than just the accuracy scores, we visualize the absolute error distributions for the different models. The histograms let us see how frequently each model produces small and larger prediction errors. The histograms are presented in Figure 9.

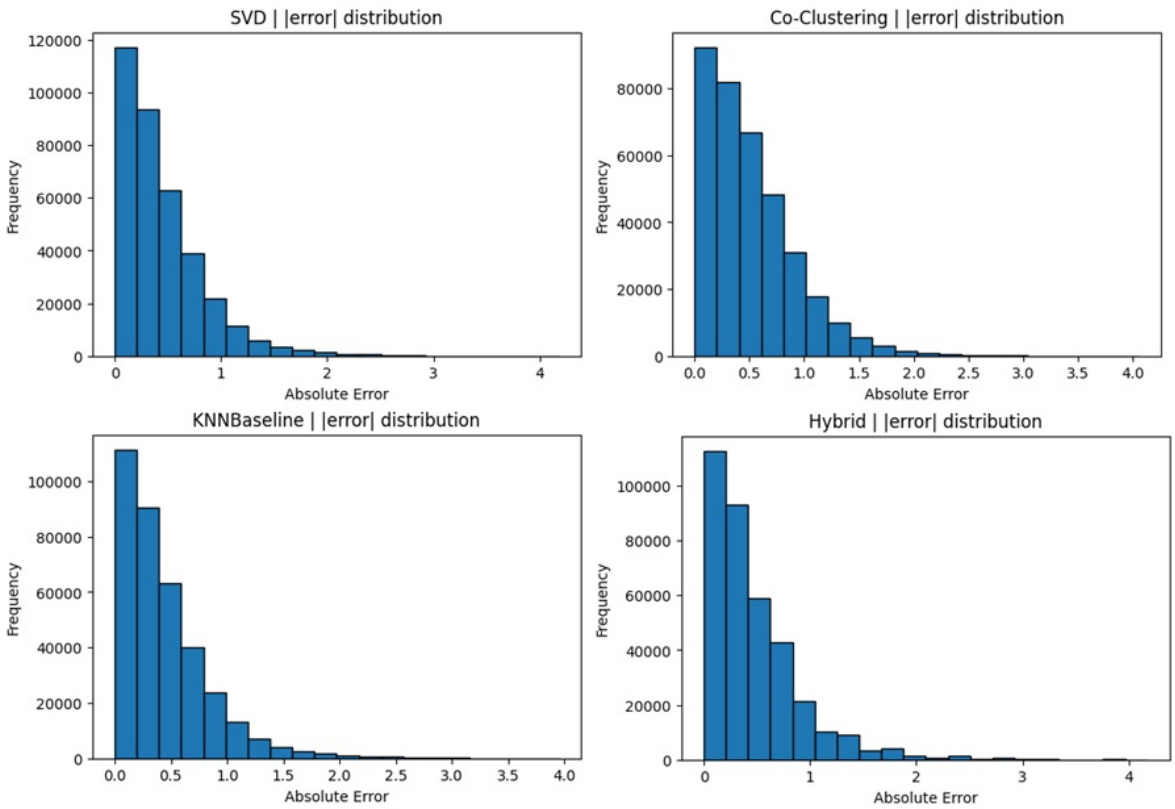


Figure 9: Absolute error distribution from all the CF and hybrid models.

SVD has the narrowest error distributions with the most predictions within an absolute error of 0-0.5. This confirms its strong regression performance.

Co-Clustering shows the widest spread, and we see a larger tail of errors above 1.0 compared to the other models. This might be because of the model's cluster structure and how it might have a tendency to make "safe" predictions in order to satisfy every user in a specific cluster. This leads to comparatively larger errors for users who are different from their assigned cluster's average behavior.

KNNBaseline is much more similar to SVD, but has a very slightly heavier tail. It has a few more errors of larger size, specifically in the 0.8-1.2 range, which can be expected since similarity-based models like KNN can struggle with the sparsity and gets noise from the nearest neighbors when handling users with little data.

The Hybrid model also displays a distribution that is very similar to SVD, which makes sense since it is partly based on SVD afterall.

5.3 Predicted Rating Behavior of the Models

We wanted a more in-depth understanding of how the models get their predictions, so we analyzed the predicted rating distributions and compared them to the true ratings that we have in the test set.

True Statistics:

- Mean true rating: 3.82
- Std: 0.71
- % true ratings ≥ 4.0: 63.26%

SVD Statistics:

- Mean predicted rating: 3.82
- Std: 0.38
- % predicted ratings ≥ 4.0: 33.34%

Co-Clustering Statistics:

- Mean predicted rating: 3.60
- Std: 0.46
- % predicted ratings ≥ 4.0: 16.39%

KNNBaseline Statistics:

- Mean predicted rating: 3.83
- Std: 0.40
- % predicted ratings ≥ 4.0: 35.39%

5.4 Overall Performance Comparison

Figure 10 provides us with a very detailed comparison of the models across all the evaluation metrics.

Comparison: Random vs Popularity vs SVD vs Co-Clustering vs KNNBaseline

Model	RMSE	MAE	Precision	Precision@5	Recall	F1	User Coverage	Conditional Coverage	ILS	Train Time (s)	Predict Time (s)
Random	0.981000	0.773000	0.634000	0.597000	0.399000	0.490000	0.930000	0.478000	0.114000	0.671000	1.523000
Popularity	1.100000	0.881000	0.637000	0.624000	0.969000	0.769000	0.999000	0.350000	0.174000	0.000000	0.000000
SVD	0.592000	0.443000	0.856000	0.722000	0.451000	0.591000	0.840000	0.418000	0.182000	24.621000	2.162000
CoClustering	0.655000	0.511000	0.889000	0.716000	0.230000	0.366000	0.639000	0.532000	0.182000	46.621000	1.782000
KNNBaseline	0.597000	0.445000	0.848000	0.720000	0.474000	0.608000	0.843000	0.360000	0.201000	66.047000	135.959000

Figure 10: Comparison table of individual models

Accuracy Metrics (RMSE & MAE):

As expected we see that both the random and popularity baselines perform significantly worse than our models which were trained on optimizing RMSE.

SVD achieves the best accuracy with RMSE = 0.592 and MAE 0.443.

It is worth noting that KNNBaseline performs almost exactly as accurately as SVD. This shows that a similarity-based approach also can model ratings well even with the high sparsity.

Co-Clustering performs significantly worse. It struggles to capture personalized user-specific preferences inside of each cluster.

Ranking Metrics (Precision, Recall, Precision@5, F1)

SVD once again leads in the ranking metrics, with a very high overall Precision (0.856) and also the best Precision@5 (0.722).

KNNBaseline performs very similarly to SVD, and even surpasses it in terms of Recall (0.474 vs 0.451).

Co-Clustering has the highest overall Precision of all models, but also by far the lowest Recall by quite a big margin, indicating that the model considers vastly fewer items to recommend to each user. This is most likely the model trying to pick “safe” picks once again because of cluster averaging.

Popularity has the highest Recall of all methods (0.969), which makes sense because it always recommends globally popular items. It seems like there is a correlation between popularity and ratings.

Coverage Metrics (User Coverage, Conditional Coverage)

Once again we see that SVD and KNNBaseline are quite similar, especially in terms of User Coverage, both covering about 84% of users. However, in Conditional Coverage we see that SVD is able to recommend a few more of its potential items. Co-Clustering has much lower User Coverage (0.639) because it rarely predicts items above the threshold. Interestingly we find that Co-Clustering has the highest Conditional Coverage (0.532), because when it does make positive predictions, they are often concentrated on a small set of consistently “safer” items.

Item-Level Diversity (ILS)

Random recommendations, as expected, have the most diverse results by far (0.114).

SVD and Co-Clustering have identical scores (0.182), which shows a decent degree of diversity.

KNNBaseline has the highest ILS (0.201), which means that it tends to recommend more similar items. This could come from KNN being based on most similar neighbors, which can lead to recommending more popular or similar items.

Efficiency Metrics (Train and Predict Time)

We see that SVD scales well, and is the fastest of our trained models. Prediction time is especially fast.

While if we look at KNN it is a very different story, and we see the biggest difference between the two so far. KNN is extremely slow, because item-item similarities must be computed when we have over 30,000 items.

Co-Clustering is faster than KNN, but still slower than SVD.

Summary

Taking all the metrics together, we can see that SVD offers the most balanced and consistently strong performance. It has top accuracy, great recommendation precision, strong coverage, acceptable diversity, and it is the best in terms of runtime and scalability. This makes SVD the most robust model.

Based on this we select SVD as our primary model for further comparison, and will evaluate it against our proposed Hybrid model in the following section.

5.5 Evaluating the Hybrid Model

SVD demonstrated that it was the most balanced and robust model among the trained models we made. However, we observed an important weakness; SVD struggled to generate recommendations for users with few ratings. As we saw in section 3.3, the majority of users fall into the low-medium activity range. If a model fails to confidently recommend items to these users, it is not sufficient enough for a finished product. In the comparison table this is briefly illustrated in the User Coverage (84%), which is good, but there are still a decent amount of users that don't get recommendations.

This motivated us to explore the possibility of a Hybrid model, which we will design to improve recommendation coverage without significantly sacrificing accuracy.

The model

The Hybrid combines popularity-based predictions for users with 11-45 reviews, and SVD predictions for users with more than 45 reviews.

This threshold was chosen because users with 11-45 reviews will often lack enough data for SVD to generate confident, personalized predictions. After further analysis we discovered that popularity-based recommendations are surprisingly strong for this user group.

Hybrid performance

The comparison table between SVD and Hybrid is shown in Figure 11.

Hybrid Model Metrics vs SVD

Model	RMSE	MAE	Precision	Precision@5	Recall	F1	User Coverage	Conditional Coverage	ILS	Train Time (s)	Predict Time (s)
Hybrid	0.647000	0.474000	0.819000	0.705000	0.490000	0.613000	0.971000	0.485000	0.165000	24.621000	2.162000
SVD	0.592000	0.443000	0.856000	0.722000	0.451000	0.591000	0.840000	0.418000	0.182000	24.621000	2.162000

Figure 11: Comparison table, Hybrid & SVD

SVD remains more accurate with lower RMSE and MAE which of course is expected because of the approach of the popularity-based model. SVD has a slightly higher Precision and Precision@5. The

Hybrid achieves higher recall which makes sense; it recommends items for more users, so it will capture more positive items overall.

In terms of coverage, we see that the Hybrid model starts to shine. User Coverage increases dramatically from 0.840 to 0.971. This shows that the Hybrid can provide at least some recommendations to almost every user. Conditional Coverage also improves from 0.418 to 0.485. This means when the Hybrid recommends items, it ends up recommending a broader subset of them. Based on this we see that the Hybrid model solved SVD's main limitation which was limited recommendations for low-medium activity users.

In terms of diversity, we see more diverse recommendations in the Hybrid model compared to SVD. Seeing ILS at 0.165 was surprising to us at first, considering the individual models previously had ILS at 0.174 and 0.182. Combining them ended up lowering ILS even more than any of them had previously achieved, which suggests that mixing SVD with popularity-based suggestions, the model gets better at avoiding repetitive and overly similar items.

Finally in terms of efficiency the train and evaluation times stay the same. The Hybrid model adds no computational cost.

Interpretation

Choosing between SVD and the Hybrid model ultimately depends on the goal of the recommender system. Both models excel in different areas, and shows strengths the other does not have.

In terms of accuracy then SVD is clearly the best choice. However, part of the reduced accuracy from the Hybrid model comes from the fact that SVD avoided some of the most difficult users, not recommending anything to about 16% of the users. Meanwhile the Hybrid model produces recommendations to these difficult users, who often can have noisy patterns in their preferences. As expected, it lowers the Hybrid accuracy metrics, but it is worth keeping in mind that it comes from making predictions the SVD model never attempted. We would like to think that recommending something is usually far better than recommending nothing. Therefore, if the goal is broad recommendations to users, the Hybrid outperforms the SVD model. It is worth mentioning that the Hybrid also offers slightly more diverse recommendations as we saw in ILS. This is central to our research question where we want to expose users to new styles.

The Hybrid model performs quite well, but it is by no means optimal. This model was made simple intentionally, by using SVD for users with rich data, and popularity-based recommendations for users with less reviews. It was more so designed to give us proof of concept for a Hybrid model potentially working, not to give us a fully optimized system that is ready to go. The results demonstrate that

hybridization works, it addresses the coverage weakness of SVD without sacrificing a considerable amount of accuracy or by requiring extra computation. This little experiment shows us that a hybrid approach combining SVD with a popularity-based component has real potential. If we were to work further on this, we would be very interested in exploring different options for an even better Hybrid model.

6. Summary of Findings

Throughout the project we have learned a lot of new things. We have experienced the challenges when working with a large and sparse dataset. The computational cost of calculating similarities, loading and visualizing the dataset requires a lot of memory and processing time. KNN and grid-search took a long time to finish because they scale so poorly with large data. However, our large dataset also allowed our models to learn more accurate and reliable patterns, as well as reducing overfitting.

We also faced the challenges of developing a recommender system on a dataset without a lot of metadata. This made it difficult for us to include content-based filtering. We noticed that taking a pure CF approach came with its problems. The cold start problem was very noticeable, because it was hard to find similar profiles for new users with few reviews and we needed an easy way to give recommendations to these users. The hybrid-approach with the popularity-based model became a solution to this. While this system might not give new users a particularly diverse set of beers, it works well for newcomers to beer-tasting by introducing them to some of the more popular beers among the community.

We have learned the advantages and disadvantages of different CF-based models. Each model offers different strengths in terms of accuracy, ranking, coverage, diversity and train/predict time. Understanding how these strengths and weaknesses would align with our goals, helped us in selecting the most appropriate one.

We believe that our recommender system will help incentivising beer-drinkers to try new beers beyond their usual preferences. We also believe that our system will help smaller brewers in becoming more competitive in an already difficult market.

7. Limitations

The evaluation is done entirely offline, which means user interaction in the real world might differ from what our metrics suggest. The systems are built purely on explicit ratings and brewery-style identifiers, which excludes possibly valuable contextual information. The dataset is highly sparse,

which can limit model performance, especially for KNN and Co-Clustering. A more dense dataset could see models improve drastically. Our Hybrid model was promising, but it is only a very simple rules-based implementation and by no means fully optimized. Neither SVD nor the Hybrid model addresses true cold-start users with zero reviews. No model was run on users with less than 11 reviews.

8. Future Work

Our Hybrid model was intentionally simple in order to provide proof of the concept working. After seeing the results, we think further experimentation could be fruitful. Perhaps using a slightly more advanced method like a learning-based approach to maintain great coverage without sacrificing precision.

In this project we only used overall rating and brewery-style together with every user. Using other attributes such as ABV could perhaps improve performance even more.

Cold-start remains a problem, and users or items with few ratings are difficult to provide recommendations to. Content-based or other hybrid approaches could be explored to address this.

As stated above we relied purely on offline evaluation. A real user study could provide more realistic feedback on how well models perform in real world scenarios.

9. References

- National Beer Wholesalers Association. (2023). *Fast facts: U.S. beer industry statistics. National Beer Wholesalers Association.* https://nbwa.org/resources/fast-facts/
- Jannach, Dietmar & Zanker, Markus & Felfernig, Alexander & Friedrich, Gerhard. (2011). *Recommender Systems.*
- Tam. A (2021, October 29) *Using singular value decomposition to build a recommender system. Machine learning mastery*. https://machinelearningmastery.com/using-singular-value-decomposition-to-build-a-recommender-system/
- Hug. N (2015). *Surprise: frequently asked questions*. Surprise documentation. https://surprise.readthedocs.io/en/stable/FAQ.html
- Hug. N (2015). *Getting started.* Surprise documentation. https://surprise.readthedocs.io/en/stable/getting_started.html#tuning-algorithm-parameters
- Jesse, M., Bauer, C. & Jannach, D. *Intra-list similarity and human diversity perceptions of recommendations: the details matter.* User Model User-Adap Inter 33, 769–802 (2023). https://doi.org/10.1007/s11257-022-09351-w
- Cañamares, R., Castells, P. & Moffat, A. *Offline evaluation options for recommender systems.* Inf Retrieval J 23, 387–410 (2020). https://doi.org/10.1007/s10791-020-09371-3